## Appendices

## Appendix A: Set-up and User Guide

These steps install and run the complete system on a Windows computer with XAMPP. The folder that holds the project is called *restaurant-ai* below; any name works if the web address is changed to match.

### A.1 Installation

1. Install **XAMPP** (Apache, MySQL, PHP 7.4 or later) and **Python 3.10 or later** (tick "Add Python to PATH" in the installer).
2. Copy the project folder to `C:\xampp\htdocs\restaurant-ai`.
3. Open the XAMPP Control Panel and start **Apache** and **MySQL**.
4. Open `http://localhost/phpmyadmin`, choose *Import*, select `db/schema.sql` and press *Go*. This creates the database `restaurant_ai`, its tables and one administrator account. If MySQL has a root password, enter it in `web/config.php`.
5. Open a terminal in the project folder and create the Python environment: `python -m venv .venv`, then `.venv\Scripts\activate`, then `pip install -r ml/requirements.txt`.
6. Place the three data files (`train.csv`, `meal_info.csv`, `fulfilment_center_info.csv`) in `data/raw`. If the file `SYNTHETIC_STAND_IN.txt` is present, delete it together with any older copies of the three files.
7. Run the pipeline, each command in the project folder: `python ml/train.py`, `python ml/evaluate_policy.py`, `python ml/error_analysis.py`, `python ml/prepare_import.py`. Always train the model on the computer on which it will be used; do not copy the trained model file between computers with different library versions.
8. Load the data into MySQL: `C:\xampp\php\php.exe web\tools\import_data.php`.
9. Start the prediction service by double-clicking `ml/start_service.bat`. Keep its window open while the site is in use.
10. Open `http://localhost/restaurant-ai/web/` and sign in as `admin` with the initial password given in the schema file. **Change the password immediately** on the *Account* page.

11. Optional: run the tests with `python tests/test_web.py http://localhost/restaurant-ai/web/` and `python tests/test_ml.py`. Set the environment variable `TEST_DB_LABEL` (for example `MySQL 8 via XAMPP`) before running the web tests so that the report records the database used.

### A.2 Using the system

- **Add or correct data:** *Records* → *Add record* (or *Edit*). Enter the week, kitchen, meal, prices, promotion flags and orders sold; optionally enter the quantity prepared and discarded. Waste is calculated when it is left empty.
- **Get a recommendation:** *Predict* → choose the kitchen and meal → check the week and prices → tick the promotion boxes that apply → choose how safe the plan should be → *Predict*. Read the recommended quantity and compare it with the usual planning line.
- **Check how good past predictions were:** *History* shows every prediction and, when the real orders have been entered, the error.
- **Add users (administrators only):** *Users* → create an account with the role *manager*.

### A.3 Troubleshooting

**Table A.1:** Common problems and their solutions

| Symptom | Likely cause and remedy |
|---|---|
| "Prediction service is not running" | Start `ml/start_service.bat`. If port 5000 is in use, set the environment variable `ML_PORT` before starting and `ML_URL` for the web server. |
| Database connection error | Check that MySQL is running in XAMPP, that the schema was imported and that the password in `web/config.php` is correct. |
| Message about cURL | Open `php.ini` in XAMPP and make sure the line `extension=curl` is not commented out; restart Apache. |
| Model file cannot be loaded | The file was trained with different library versions. Run `python ml/train.py` again on this computer. |
| Time shown is wrong | Change `timezone` in `web/config.php`. |

## Appendix B: Automated Test Cases

Web application tests (tests/test_web.py), run on 2026-10-21 19:05; database: MySQL (XAMPP). Result: 48 of 48 passed.

**Table B.1:** All automated web-application checks

| ID | Check | Result |
|---|---|---|
| T01 | Dashboard redirects to login when not signed in | Pass |
| T02 | records.php requires login | Pass |
| T02 | predict.php requires login | Pass |
| T02 | predictions.php requires login | Pass |
| T02 | model.php requires login | Pass |
| T02 | users.php requires login | Pass |
| T02 | record_edit.php requires login | Pass |
| T03 | report images require login | Pass |
| T04 | Wrong password is rejected | Pass |
| T05 | SQL injection in the username does not log in | Pass |
| T06 | Unknown user gets the same message as a wrong password | Pass |
| T07 | Correct login redirects to the dashboard | Pass |
| T08 | Dashboard loads with KPIs and charts | Pass |
| T09 | Session cookie is HttpOnly | Pass |
| T10 | Records list shows data | Pass |
| T11 | Negative orders are rejected | Pass |
| T12 | Waste larger than prepared is rejected | Pass |
| T13 | Non-numeric price is rejected | Pass |
| T14 | Form without a valid CSRF token is refused (HTTP 400) | Pass |
| T15 | Valid record is saved | Pass |
| T16 | New record appears in the list; waste auto-calculated as prepared - sold (= 40) | Pass |
| T17 | Duplicate week for the same meal/kitchen is refused | Pass |
| T18 | Prediction form loads with pre-filled values | Pass |
| T19 | Prediction is produced and shown | Pass |
| T20 | Predicted value is a sensible positive number | Pass |
| T21 | Prediction for a week with no history gives a clear error | Pass |
| T22 | Prediction with an invalid price is rejected | Pass |
| T23 | Prediction form without valid CSRF token is refused | Pass |
| T24 | Predicting a week with known actual orders shows the actual value | Pass |
| T25 | Prediction history lists predictions with error against actuals | Pass |
| T26 | Model & results page loads | Pass |
| T27 | Report image is served as PNG | Pass |
| T28 | Path traversal on the image script is blocked | Pass |
| T29 | CSV export works | Pass |
| T30 | Odd query-string values do not break the list | Pass |
| T31 | Prediction service /health reports the model | Pass |
| T32 | Prediction service rejects malformed input with 4xx (not a crash) | Pass |
| T33 | Weak password is rejected when creating a user | Pass |
| T34 | Admin can create a manager | Pass |

| ID | Check | Result |
|---|---|---|
| T35 | Script tags in a name are HTML-escaped (XSS blocked) | Pass |
| T36 | New manager can log in | Pass |
| T37 | Manager cannot open the Users page (403) | Pass |
| T38 | Manager sees no Delete buttons | Pass |
| T39 | Manager cannot delete records even by sending the request directly (403) | Pass |
| T40 | Account locks after 5 wrong passwords (right password refused) | Pass |
| T41 | Admin can delete a record | Pass |
| T42 | Temporary user removed | Pass |
| T43 | After logout the dashboard is protected again | Pass |

Machine-learning tests (tests/test_ml.py), run on 2026-09-22 02:13. Result: 13 of 13 passed.

**Table B.2:** All automated machine-learning checks

| ID | Check | Result |
|---|---|---|
| M01 | lag_1 of week 5 is week 4 (=40); lag_2 of week 5 is week 3 (missing -> NaN) | Pass |
| M02 | week 4 has lag_1 missing (week 3 absent), not the wrong value 20 | Pass |
| M03 | Features of a week do not change when that week's and later orders are changed (no leakage) | Pass |
| M04 | Train, validation and test weeks do not overlap and are in time order | Pass |
| M05 | Safety factors increase with the service level | Pass |
| M06 | Safety factors are between 0.5 and 3 | Pass |
| M07 | waste - shortage = prepared - demand (identity holds) | Pass |
| M08 | waste = 50 (10 + 0 + 0 + 40) and shortage = 20 for the hand-made example | Pass |
| M09 | Web service answered all 150 sampled requests | Pass |
| M10 | Web-service prediction equals the offline model prediction (max difference < 0.1 portion) | Pass |
| M11 | Service refuses a prediction with no sales history (HTTP 422) | Pass |
| M12 | Service refuses a non-JSON body (HTTP 400) | Pass |
| M13 | Acceptance: chosen model (Gradient boosting) has lower test WAPE than the best baseline | Pass |

## Appendix C: Selected Source Code

The excerpts below are copied directly from the source files by the program that builds this document.

### C.1 Feature engineering (ml/features.py)

```
def add_lag_features(df: pd.DataFrame) -> pd.DataFrame:
    """
    Add 'orders k weeks ago' columns.

    We look the lag up by WEEK NUMBER (not by row position) because some
    weeks are missing for some meal/centre pairs. Shifting by row position
    would silently pick the wrong week.
    """
    out = df.copy()
    for k in LAGS:
        past = df[KEYS + ["num_orders"]].copy()
        past["week"] = past["week"] + k          # week 10's orders become "week 11, lag
   k=1"
        past = past.rename(columns={"num_orders": f"lag_{k}"})
        out = out.merge(past, on=KEYS, how="left")

    out["lag_mean_2"] = out[["lag_1", "lag_2"]].mean(axis=1)          # NaN ignored
    out["lag_mean_4"] = out[[f"lag_{k}" for k in LAGS]].mean(axis=1)
    # Is demand rising or falling? (last week vs the average of the older weeks)
    # Values > 1 mean last week was busier than the weeks before it.
    older = out[["lag_2", "lag_3", "lag_4"]].mean(axis=1)
    out["lag_trend"] = out["lag_1"] / older
    return out


def build_features(orders: pd.DataFrame, meals: pd.DataFrame, centers: pd.DataFrame) ->
   pd.DataFrame:
    """
    orders  : rows with week, center_id, meal_id, checkout_price, base_price,
              emailer_for_promotion, homepage_featured, num_orders
              (num_orders may be NaN for the week we want to predict)
    Returns the same rows with all model input columns added.
    """
    df = orders.merge(meals, on="meal_id", how="left").merge(centers, on="center_id",
   how="left")

    df["price_ratio"] = df["checkout_price"] / df["base_price"]       # < 1 means the meal
   is discounted
    df["discount_pct"] = (1 - df["price_ratio"]) * 100

    # Encode "which week of the year" as a point on a circle (sine + cosine) instead
    # of a single number 1-52. A single number would make the model think week 52
    # and week 1 are far apart, when they're actually next to each other (both near
    # the end/start of the year); sine+cosine wrap around correctly.
    week_of_year = (df["week"] - 1) % 52
    df["week_sin"] = np.sin(2 * np.pi * week_of_year / 52)
    df["week_cos"] = np.cos(2 * np.pi * week_of_year / 52)

    df = add_lag_features(df)
    # Categorical columns (meal category, cuisine, kitchen type/region/city) are
    # treated as LABELS, not numbers -> store them as text so scikit-learn one-hot
    # encodes them, rather than wrongly treating e.g. category "3" as bigger than "1".
    for col in CATEGORICAL_FEATURES:
        df[col] = df[col].astype(str)
    return df
```

### C.2 Prediction endpoint (ml/service.py)

```
@app.post("/predict")
def predict():
    p = request.get_json(silent=True)
```

```
    if not isinstance(p, dict):
        return error("Request body must be JSON.", 400)
    try:
        week = require(p, "week", int)
        price = require(p, "checkout_price", float)
        base = require(p, "base_price", float)
        promo = require(p, "emailer_for_promotion", int)
        featured = require(p, "homepage_featured", int)
        center = {"center_id": require(p, "center.center_id", int),
                  "city_code": require(p, "center.city_code", str),
                  "region_code": require(p, "center.region_code", str),
                  "center_type": require(p, "center.center_type", str),
                  "op_area": require(p, "center.op_area", float)}
        meal = {"meal_id": require(p, "meal.meal_id", int),
                "category": require(p, "meal.category", str),
                "cuisine": require(p, "meal.cuisine", str)}
    except ValueError as e:
        return error(str(e))

    if price <= 0 or base <= 0:
        return error("Prices must be greater than zero.")
    if promo not in (0, 1) or featured not in (0, 1):
        return error("Promotion and featured flags must be 0 or 1.")

    # ---- the last 4 weeks of sales for this meal in this kitchen ----------------
    history = p.get("history", [])
    if not isinstance(history, list):
        return error("history must be a list.")
    hist_rows = []
    for h in history:
        try:
            hw, ho = int(h["week"]), float(h["num_orders"])
        except (KeyError, TypeError, ValueError):
            return error("Each history item needs week and num_orders.")
        # Only keep history that's actually usable: strictly before the week we're
        # predicting, and no older than the furthest-back lag the model uses.
        if week - max(LAGS) <= hw < week and ho >= 0:
            hist_rows.append((hw, ho))
    if not hist_rows:
        # Mirrors the rule in has_some_history() (features.py): the web app must
        # refuse a prediction rather than silently guess from nothing.
        return error(f"No sales history found for the {max(LAGS)} weeks before week {week}.
"
                     "Enter recent weekly sales first.")

    # Build a tiny table that looks exactly like a slice of the training data: the
    # past weeks (with their real order counts) plus one row for the week we want
    # to predict, with num_orders left blank. Then call the SAME build_features()
    # function used by train.py. This is what guarantees "no training/serving
    # skew" (Section 3.3.5): the lag features, price ratio, seasonality etc. are
    # computed by identical code whether the model is being trained offline or
    # answering a live request here.
    rows = [{"week": hw, "center_id": center["center_id"], "meal_id": meal["meal_id"],
             "checkout_price": price, "base_price": base, "emailer_for_promotion": 0,
             "homepage_featured": 0, "num_orders": ho} for hw, ho in hist_rows]
    rows.append({"week": week, "center_id": center["center_id"], "meal_id":
meal["meal_id"],
                 "checkout_price": price, "base_price": base, "emailer_for_promotion":
promo,
                 "homepage_featured": featured, "num_orders": np.nan})   # <- the week we
predict

    meals = pd.DataFrame([meal])
    centers = pd.DataFrame([center])
    feats = build_features(pd.DataFrame(rows), meals, centers)
    target_row = feats[feats["week"] == week]       # pull out just the row we actually
want a prediction for
    predicted = float(np.clip(MODEL.predict(target_row[ALL_FEATURES])[0], 0, None))

    # Work out the recommended quantity at every service level (50/70/80/90/95%),
    # not just the one the user asked for, so the web page can show all of them
    # side by side and let the user compare.
    levels = {}
    for level, factor in ARTEFACT["safety_factors"].items():
        levels[level] = {"safety_factor": factor, "recommended_prep": int(round(predicted *
factor))}
```

```
    chosen = str(p.get("service_level", ARTEFACT["default_service_level"]))
    if chosen not in levels:
        chosen = str(ARTEFACT["default_service_level"])

    # Simple "how sure are we" hint: fewer weeks of history = less reliable. This
    # is a plain rule of thumb for the UI, not a statistical confidence interval.
    weeks_used = len(hist_rows)
    reliability = "high" if weeks_used >= 4 else "medium" if weeks_used >= 2 else "low"

    return jsonify({
        "ok": True,
        "predicted_demand": round(predicted, 1),
        "service_level": chosen,
        "recommended_prep": levels[chosen]["recommended_prep"],
        "all_levels": levels,
        "history_weeks_used": weeks_used,
        "reliability": reliability,
        "model_name": ARTEFACT["model_name"],
        "trained_on_synthetic_data": ARTEFACT["trained_on_synthetic_data"],
    })
```

## C.3 Matching the coverage of the manual plan (ml/evaluate_policy.py)

```
    pred = test["predicted"].to_numpy()

    def coverage(f):
        return float((pred * f >= actual).mean())

    def factor_for_coverage(target):
        # Binary search (bisection) for the safety factor that gives exactly the
        # manual policy's coverage: coverage is a steadily increasing function of
        # the factor (a bigger factor means preparing more, which can only cover
        # more cases, never fewer), so we can narrow in on it by repeatedly
        # halving the search range. 60 rounds is far more precision than needed;
        # it converges in well under 30.
        lo, hi = 0.3, 5.0                       # coverage grows as the factor grows
        for _ in range(60):
            mid = (lo + hi) / 2
            if coverage(mid) < target:
                lo = mid
            else:
                hi = mid
        return hi
```

## C.4 Login with lock-out (web/includes/auth.php)

```
function attempt_login(string $username, string $password): ?string
{
    $generic = 'Wrong username or password.';
    $user = db_one('SELECT * FROM users WHERE username = ?', [$username]);

    if ($user === null) {
        // Even though we already know this username doesn't exist, we still run
        // password_verify() against a dummy hash before returning. Checking a
        // password normally takes a small, measurable amount of time; without
        // this line, a real username would take noticeably longer to reject than
        // a fake one, and an attacker could use that timing difference to guess
        // which usernames exist on the system ("timing attack").
        password_verify($password,
    '$2y$12$daZPQOOE4BZfpe5fPAwXm.sLILQ0l.NBxHDHnrYjUKZ8KmBZWkNom'); // same time cost
        return $generic;
    }

    if ($user['locked_until'] !== null && strtotime($user['locked_until']) > time()) {
        return 'Too many failed attempts. Try again in a few minutes.';
    }

    if (!password_verify($password, $user['password_hash'])) {
        $fails = (int)$user['failed_logins'] + 1;
        $lock = null;
```

```
        if ($fails >= MAX_FAILED_LOGINS) {
            // 5 wrong passwords in a row -> lock the account for 5 minutes and
            // reset the counter, so the next failed attempt (after the lock
            // expires) starts counting from 1 again, not 6.
            $lock = date('Y-m-d H:i:s', time() + LOCK_MINUTES * 60);
            $fails = 0;
        }
        db_run('UPDATE users SET failed_logins = ?, locked_until = ? WHERE user_id = ?',
            [$fails, $lock, $user['user_id']]);
        return $generic;
    }

    db_run('UPDATE users SET failed_logins = 0, locked_until = NULL WHERE user_id = ?',
    [$user['user_id']]);
    session_regenerate_id(true);
    $_SESSION['user'] = [
        'user_id'   => (int)$user['user_id'],
        'username'  => $user['username'],
        'full_name' => $user['full_name'],
        'role'      => $user['role'],
    ];
    return null;
}
```

## Appendix D: Database Schema

The schema below is the content of `db/schema.sql` without the statement that creates the first administrator account.

```
-- ============================================================================
-- schema.sql : MySQL database for the Restaurant Food Demand & Waste system
-- Import with phpMyAdmin (Import tab) or:  mysql -u root < db/schema.sql
-- Works on MySQL 5.7+/8.x and MariaDB 10.3+ (XAMPP).
-- ============================================================================

CREATE DATABASE IF NOT EXISTS restaurant_ai
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE restaurant_ai;

-- Drop in child-first order so the script can be re-run safely while developing.
DROP TABLE IF EXISTS predictions;
DROP TABLE IF EXISTS weekly_records;
DROP TABLE IF EXISTS meals;
DROP TABLE IF EXISTS centers;
DROP TABLE IF EXISTS users;

-- ----------------------------------------------------------------------------
-- users : people who can log in
--
-- All tables use ENGINE=InnoDB rather than MySQL's older MyISAM engine,
-- because InnoDB supports foreign keys (so a record can't point at a
-- center/meal/user that doesn't exist) and transactions (so a multi-step
-- change, e.g. the bulk CSV import, either fully succeeds or fully fails,
-- never leaves the database half-written).
-- ----------------------------------------------------------------------------
CREATE TABLE users (
    user_id        INT UNSIGNED NOT NULL AUTO_INCREMENT,
    username       VARCHAR(50)  NOT NULL,
    password_hash  VARCHAR(255) NOT NULL,          -- bcrypt hash, never the password
    itself
    full_name      VARCHAR(100) NOT NULL,
    role           ENUM('admin','manager') NOT NULL DEFAULT 'manager', -- manager = normal
    user; admin can also manage accounts
    failed_logins  TINYINT UNSIGNED NOT NULL DEFAULT 0,  -- reset to 0 on a successful
    login
    locked_until   DATETIME NULL,                  -- temporary lock after too many wrong
    passwords; NULL = not locked
    created_at     DATETIME NOT NULL,
    PRIMARY KEY (user_id),
    UNIQUE KEY uq_users_username (username)         -- no two accounts can share a username
) ENGINE=InnoDB;

-- ----------------------------------------------------------------------------
-- centers : a kitchen / outlet (called "fulfilment center" in the public dataset)
-- ----------------------------------------------------------------------------
CREATE TABLE centers (
    center_id    INT NOT NULL,
    city_code    VARCHAR(10) NOT NULL,
    region_code  VARCHAR(10) NOT NULL,
    center_type  VARCHAR(10) NOT NULL,
    op_area      DECIMAL(4,1) NOT NULL,            -- operating area (size of the kitchen)
    PRIMARY KEY (center_id)
) ENGINE=InnoDB;

-- ----------------------------------------------------------------------------
-- meals : a menu item
-- ----------------------------------------------------------------------------
CREATE TABLE meals (
    meal_id   INT NOT NULL,
    category  VARCHAR(40) NOT NULL,
    cuisine   VARCHAR(40) NOT NULL,
    PRIMARY KEY (meal_id)
) ENGINE=InnoDB;

-- ----------------------------------------------------------------------------
-- weekly_records : one row = one meal, at one center, in one week
-- This is the main table; the model learns from it.
```

```
-- ----------------------------------------------------------------------------
CREATE TABLE weekly_records (
    record_id              INT UNSIGNED NOT NULL AUTO_INCREMENT,
    week_no                SMALLINT UNSIGNED NOT NULL,
    center_id              INT NOT NULL,
    meal_id                INT NOT NULL,
    checkout_price         DECIMAL(8,2) NOT NULL,
    base_price             DECIMAL(8,2) NOT NULL,
    emailer_for_promotion  TINYINT(1) NOT NULL DEFAULT 0,   -- 1 = promoted by e-mail that
    week
    homepage_featured      TINYINT(1) NOT NULL DEFAULT 0,   -- 1 = featured on the homepage
    orders_sold            INT UNSIGNED NOT NULL,           -- actual demand (portions)
    prepared_qty           INT UNSIGNED NULL,               -- portions cooked/prepared
    waste_qty              INT UNSIGNED NULL,               -- portions thrown away
    waste_source           ENUM('recorded','simulated') NULL, -- see note below
    created_by             INT UNSIGNED NULL,
    created_at             DATETIME NOT NULL,
    PRIMARY KEY (record_id),
    UNIQUE KEY uq_record (week_no, center_id, meal_id),     -- no duplicate week for the
    same meal/center
    -- This index exists specifically so "give me this meal's last 4 weeks at this
    -- center" (what the prediction screen needs every time) is a fast index lookup
    -- instead of a full table scan over 450,000+ rows.
    KEY idx_record_pair (center_id, meal_id, week_no),
    CONSTRAINT fk_rec_center FOREIGN KEY (center_id) REFERENCES centers (center_id),
    CONSTRAINT fk_rec_meal   FOREIGN KEY (meal_id)   REFERENCES meals (meal_id),
    -- If the user who entered a record is later deleted, keep the record but clear
    -- who entered it, rather than deleting their historical data.
    CONSTRAINT fk_rec_user   FOREIGN KEY (created_by) REFERENCES users (user_id) ON DELETE
    SET NULL
) ENGINE=InnoDB;
-- NOTE on waste_source: the public dataset contains orders only. For those rows
-- prepared_qty/waste_qty are SIMULATED (see thesis, Methodology) and tagged
-- 'simulated'. Rows typed in through the website are tagged 'recorded'.

-- ----------------------------------------------------------------------------
-- predictions : every forecast the system produced (an audit trail)
-- ----------------------------------------------------------------------------
CREATE TABLE predictions (
    prediction_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
    week_no            SMALLINT UNSIGNED NOT NULL,
    center_id          INT NOT NULL,
    meal_id            INT NOT NULL,
    checkout_price     DECIMAL(8,2) NOT NULL,
    base_price         DECIMAL(8,2) NOT NULL,
    emailer_for_promotion TINYINT(1) NOT NULL DEFAULT 0,
    homepage_featured  TINYINT(1) NOT NULL DEFAULT 0,
    predicted_demand   DECIMAL(10,1) NOT NULL,          -- the raw model output, before the
    safety factor is applied
    service_level      DECIMAL(3,2) NOT NULL,           -- e.g. 0.80 = cover demand ~80 %
    of the time
    recommended_prep   INT UNSIGNED NOT NULL,           -- predicted_demand x safety
    factor, rounded up: what to actually cook
    model_name         VARCHAR(60) NOT NULL,            -- which model produced this (kept
    even if a different model is chosen later)
    history_weeks_used TINYINT UNSIGNED NOT NULL,       -- how many of the last 4 weeks
    were actually available
    created_by         INT UNSIGNED NULL,
    created_at         DATETIME NOT NULL,
    PRIMARY KEY (prediction_id),
    KEY idx_pred_pair (center_id, meal_id, week_no),
    CONSTRAINT fk_pred_center FOREIGN KEY (center_id) REFERENCES centers (center_id),
    CONSTRAINT fk_pred_meal   FOREIGN KEY (meal_id)   REFERENCES meals (meal_id),
    CONSTRAINT fk_pred_user   FOREIGN KEY (created_by) REFERENCES users (user_id) ON DELETE
    SET NULL
) ENGINE=InnoDB;
```

## Appendix E: Additional Results

### E.1 Full accuracy figures

**Table E.1:** Accuracy measures of all approaches on the validation and test weeks

| Approach | Weeks | MAE | RMSE | WAPE % | RMSLE | $R^2$ | Bias % |
|---|---|---|---|---|---|---|---|
| Baseline: same as last week | Validation | 119.53 | 345.54 | 45.22 | 0.7075 | 0.3282 | -0.24 |
| Baseline: same as last week | Test | 79.36 | 136.33 | 34.80 | 0.6691 | 0.7575 | +1.46 |
| Baseline: 4-week average | Validation | 121.01 | 335.41 | 45.78 | 0.6681 | 0.3670 | -1.25 |
| Baseline: 4-week average | Test | 81.95 | 162.89 | 35.94 | 0.6267 | 0.6538 | +4.26 |
| Ridge regression | Validation | 144.00 | 2123.56 | 54.48 | 0.6150 | -24.3727 | +0.26 |
| Ridge regression | Test | 92.43 | 314.48 | 40.53 | 0.6130 | -0.2904 | -11.03 |
| Random forest | Validation | 79.91 | 214.31 | 30.23 | 0.4969 | 0.7416 | -10.23 |
| Random forest | Test | 63.94 | 110.42 | 28.04 | 0.5001 | 0.8409 | -9.30 |
| Gradient boosting | Validation | 78.79 | 211.10 | 29.81 | 0.4847 | 0.7493 | -10.76 |
| Gradient boosting | Test | 63.71 | 110.46 | 27.94 | 0.4878 | 0.8408 | -9.91 |

### E.2 Safety factors

**Table E.2:** Safety factors learned on the validation weeks

| Target service level | Safety factor |
|---|---|
| 50% | 1.0561 |
| 70% | 1.3160 |
| 80% | 1.5092 |
| 90% | 1.8214 |
| 95% | 2.1580 |

### E.3 Test error by meal category and by kitchen type

**Table E.3:** Test error by meal category and kitchen type

| Group | Rows | Share of demand % | WAPE % | Bias % |
|---|---|---|---|---|
| Beverages | 8,978 | 34.2 | 24.38 | -9.42 |
| Biryani | 1,437 | 0.7 | 44.90 | -17.86 |
| Desert | 2,113 | 2.2 | 38.88 | -15.52 |
| Extras | 993 | 3.6 | 24.38 | -2.45 |
| Fish | 1,369 | 1.8 | 37.94 | -18.49 |
| Other Snacks | 2,088 | 4.3 | 29.79 | -11.79 |
| Pasta | 1,915 | 1.5 | 44.47 | -14.65 |
| Pizza | 2,306 | 8.4 | 34.76 | -21.02 |
| Rice Bowl | 2,310 | 16.4 | 25.25 | -5.41 |
| Salad | 2,310 | 9.6 | 27.00 | -0.92 |
| Sandwich | 2,306 | 11.8 | 28.37 | -14.65 |
| Seafood | 1,671 | 1.5 | 43.32 | -16.93 |

| Group | Rows | Share of demand % | WAPE % | Bias % |
|---|---|---|---|---|
| Soup | 909 | 1.4 | 29.75 | -4.70 |
| Starters | 1,993 | 2.7 | 32.73 | -10.88 |
| Kitchen type TYPE_A | 18,837 | 56.4 | 27.65 | -9.64 |
| Kitchen type TYPE_B | 6,886 | 26.9 | 25.84 | -10.34 |
| Kitchen type TYPE_C | 6,975 | 16.8 | 32.25 | -10.15 |

## E.4 Weekly demand over time

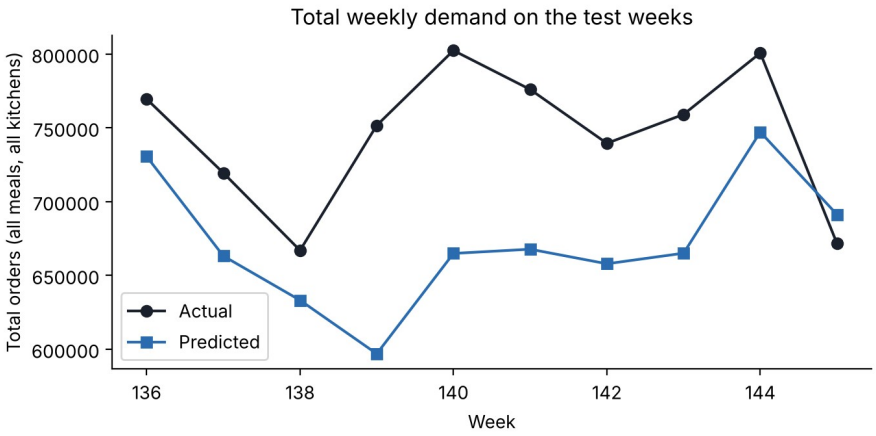



**Figure E.1:** Total weekly orders over all kitchens and meals

## Appendix F: Supplementary Tables

These tables support the main chapters but are reproduced here in full to keep the main document within the word limit.

### F.1 Literature reviewed

**Table F.1:** Summary of the most relevant related work

| Source | Focus | Relevance to this project |
|---|---|---|
| UNEP (2024) | Global food waste estimate by sector; measurement guidance. | Establishes the scale: ~28% of wasted food is from food service, motivating prevention at source. |
| Betz et al. (2015) | Magnitude of food waste in the Swiss food-service industry and its potential for reduction. | Quantified evidence of the scope to reduce waste in food service. |
| Arunraj and Ahrens (2015) | Hybrid SARIMA and quantile regression for daily banana sales at a German discount retailer. | Forecasts quantiles rather than averages to support stocking; outperformed SARIMA and neural-network benchmarks. |
| Tsoumakas (2019) | Survey of machine-learning techniques for food sales prediction. | Maps design decisions (granularity, inputs, outputs, metrics); short-term prediction can reduce waste and stock-outs. |
| Huber and Stuckenschmidt (2020) | Daily demand forecasting for a bakery chain, with emphasis on special calendar days. | Machine learning outperforms traditional approaches for perishable goods. |
| Malefors et al. (2022) | Guest-attendance forecasting for Swedish school and preschool kitchens using random forest and neural networks. | Random forest performed best, with estimated economic savings over a no-plan baseline; harder during extreme events. |
| Rodrigues et al. (2024) | Machine-learning models for short-term demand forecasting in food catering services. | Directly relevant: frames catering demand forecasting as a means of reducing food waste. |
| Makridakis et al. (2022) | M5 retail forecasting competition results. | Evidence that tree-based boosting is highly competitive for large retail-demand panels. |

### F.2 Design-science guidelines

**Table F.2:** Design-science guidelines and their application in this project

| Guideline (Hevner et al., 2004) | Application |
|---|---|
| 1. Design as an artefact | A working system: MySQL database, prediction service and web interface (Chapter 4). |
| 2. Problem relevance | Over-preparation and food waste in food service (Section 1.1). |
| 3. Design evaluation | Automated functional tests, model evaluation on unseen weeks, waste study, usability review (Section 4.5, Chapter 5). |
| 4. Research contributions | Leakage-free evaluation, equal-service waste comparison, documented waste-simulation assumptions (Section 1.4). |
| 5. Research rigour | Time-ordered split, single use of the test set, fixed random seeds, recorded software versions (Section 3.3.5). |
| 6. Design as a search process | Five candidate models compared; features and safety factors chosen on validation data only (Section 3.3.2). |

| Guideline (Hevner et al., 2004) | Application |
|---|---|
| 7. Communication | This thesis, the set-up guide and the source code (shared project folder). |

## F.3 Software and technologies used

**Table F.3:** Software used in the project

| Software | Version / role |
|---|---|
| XAMPP | Apache, MySQL and PHP bundle used for local development and testing. |
| PHP | 7.4 or later — the web application. |
| MySQL (InnoDB) | Persistent storage; required by the project contract. |
| Python 3.10+ | Data pipeline, model training and the prediction service. |
| scikit-learn | Ridge regression, random forest and gradient boosting models. |
| pandas / NumPy | Data loading, cleaning and feature engineering. |
| Flask | The small web API that serves predictions. |
| VS Code | Code editor used throughout development. |

## F.4 Usability heuristic self-assessment

Self-assessment of the interface against Nielsen's ten usability heuristics (Nielsen, 1994).

**Table F.4:** Self-assessment against Nielsen's usability heuristics

| Heuristic | How the interface addresses it |
|---|---|
| 1. Visibility of system status | Dashboard KPIs, prediction history and a loading state on the Predict page. |
| 2. Match with the real world | Plain field names (kitchen, meal, week) rather than internal IDs where practical. |
| 3. User control and freedom | Edit and cancel are available on records; no destructive action without confirmation. |
| 4. Consistency and standards | The same layout, navigation and button styles are used on every page. |
| 5. Error prevention | Server-side validation rejects negative orders, waste exceeding prepared quantity and invalid prices. |
| 6. Recognition rather than recall | Dropdowns list known kitchens and meals; recent values are pre-filled where sensible. |
| 7. Flexibility and efficiency of use | Five safety levels let an experienced user skip straight to the plan they want. |
| 8. Aesthetic and minimalist design | Pages show only the fields needed for the task at hand. |
| 9. Help users recognise and recover from errors | Validation messages name the field and the problem in plain language. |
| 10. Help and documentation | The set-up and user guide (Appendix A) covers every screen. |

## F.5 Functional requirements (full table)

**Table F.5:** Functional requirements

| ID | Requirement | Verified by |
|---|---|---|
| FR1 | Users sign in with a user name and password and can sign out. | T01–T04, T06, T07, T43 |
| FR2 | Two roles; managers work with data and predictions, only admins manage users and delete records. | T37–T39, T41 |
| FR3 | The database stores kitchens, meals, weekly records (orders, prices, promotions, prepared/waste) and predictions. | Schema; T15, T16 |
| FR4 | Weekly records are validated on entry/edit; negative values, excess waste and duplicate weeks are refused. | T11–T17 |
| FR5 | Records can be filtered, paged and exported to a CSV file. | T10, T29, T30 |
| FR6 | The user requests a prediction for a kitchen, meal and week, entering planned prices and promotions. | T18–T23 |
| FR7 | The result shows predicted demand, recommended preparation per service level, the usual plan and a chart. | T19, T20 |
| FR8 | Every prediction is saved and, once real orders are entered, compared with them. | T24, T25 |
| FR9 | A dashboard shows key figures and charts, optionally for one kitchen. | T08 |
| FR10 | A results page shows the model comparison and waste study. | T26, T27 |
| FR11 | Admins create users with acceptable passwords; every user can change their own. | T33, T34, T36 |
| FR12 | Historical data can be bulk-loaded into the database from prepared files. | Import tool |

## F.6 Non-functional requirements (full table)

**Table F.6:** Non-functional requirements

| Quality | Requirement | Verified by |
|---|---|---|
| NFR1 Security | Protect against the main web threats (injection, broken access control, weak authentication, XSS, CSRF). | T05, T14, T28, T35, T37–T40 |
| NFR2 Usability | Plain language; a prediction in at most three steps (choose meal, confirm, read result). | Appendix F.4 |
| NFR3 Performance | Returned well under one second; measured median 22.3 ms. | Section 4.4.3 |
| NFR4 Portability | Runs on a standard XAMPP Windows install (Apache, MySQL, PHP 7.4+) with Python for the model. | Appendix A |
| NFR5 Maintainability | One feature-building function for training and serving; settings in one config file per part. | M10 |
| NFR6 Reliability | If the prediction service is down the user sees a clear message and no data is lost. | Manual check; T21, T32, M11, M12 |
| NFR7 Data integrity | Foreign keys, a uniqueness rule on (week, kitchen, meal) and transactional bulk import. | T17; schema |
| NFR8 Reproducibility | Results regenerate from raw data by running the scripts; seeds and versions recorded. | Section 3.3.5 |

## F.7 System components (full table)

**Table F.7:** Main components of the system

| Component | Responsibility |
|---|---|
| Web browser | Interface for managers and administrators (HTML forms, CSS, small JavaScript charts). |
| Web application (PHP) | Pages for login, dashboard, records, prediction, history, results and users; shared authentication, database (PDO) and layout code. |
| Security layer | Session handling, password hashing, login lock-out, CSRF tokens, role checks, output escaping. |
| MySQL database | Tables users, centers (kitchens), meals, weekly_records and predictions. |
| Prediction service (Flask) | Endpoints /health and /predict; validates input, builds features, runs the model and returns results. |
| Trained model file | A scikit-learn pipeline, safety factors and metadata, saved as one file. |
| Offline pipeline | Scripts: data loading, training and selection, waste study, error analysis and import preparation. |
| Result files | JSON files and PNG charts that the Results page displays. |

## F.8 Security threats, controls and tests (full table)

**Table F.8:** Threats, controls and tests

| Threat | Control implemented | Test |
|---|---|---|
| SQL injection | All queries use PDO prepared statements; no user input is joined into SQL text. | T05 |
| Broken authentication | Bcrypt-hashed passwords; sessions renewed at login; HttpOnly/SameSite cookies; same error and timing for unknown user and wrong password; five-minute lock-out after five attempts. | T04, T06, T07, T09, T33, T40 |
| Broken access control | Every page checks login; admin-only pages/actions also check the role server-side. | T02, T03, T37–T39 |
| Cross-site request forgery | A secret token in every form, checked on every state-changing request; logout is a POST. | T14, T23 |
| Cross-site scripting | All output is HTML-escaped through one helper function. | T35 |
| Path traversal | The report-image script accepts only file names from a fixed whitelist. | T28 |
| Exposure of the model service | Listens only on 127.0.0.1, can require a secret key; input is validated strictly. | Manual check; T21, T32, M11, M12 |
| Default credentials | A default admin is created by the schema; the set-up guide requires the password changed at first login. | Appendix A |

## F.9 Test error by situation (full table)

**Table F.9:** Test error by situation

| Situation | Group | Rows | Share of demand % | WAPE % | Bias % |
|---|---|---|---|---|---|
| E-mail promotion | e-mail promotion | 1,722 | 8.7 | 33.88 | -13.88 |
| E-mail promotion | no e-mail promotion | 30,976 | 91.3 | 27.37 | -9.53 |
| Home-page feature | homepage featured | 2,683 | 14.7 | 28.18 | -11.65 |

| Situation | Group | Rows | Share of demand % | WAPE % | Bias % |
|---|---|---|---|---|---|
| Home-page feature | not featured | 30,015 | 85.3 | 27.90 | -9.62 |
| Price | discounted (>1%) | 7,561 | 25.6 | 31.66 | -11.41 |
| Price | full price | 25,137 | 74.4 | 26.66 | -9.40 |
| Demand size | low demand (lowest third) | 11,369 | 6.1 | 53.94 | +27.90 |
| Demand size | medium demand | 10,602 | 20.2 | 33.56 | -4.52 |
| Demand size | high demand (highest third) | 10,727 | 73.8 | 24.27 | -14.49 |
| Recent trend | last week > 20% above average | 8,272 | 25.1 | 29.65 | -12.18 |
| Recent trend | last week > 20% below average | 9,936 | 21.9 | 32.94 | -10.42 |
| Recent trend | stable recent demand | 14,490 | 52.9 | 25.06 | -8.63 |

## F.10 Size of the code base

**Table F.10:** Size of the code base (lines)

| Part | Lines |
|---|---|
| Machine-learning pipeline and service (Python) | 1,060 |
| Web application (PHP) | 2,010 |
| Interface assets (CSS, JavaScript) | 235 |
| Database (SQL) | 129 |
| Automated tests (Python) | 424 |
| **Total** | **3,858** |